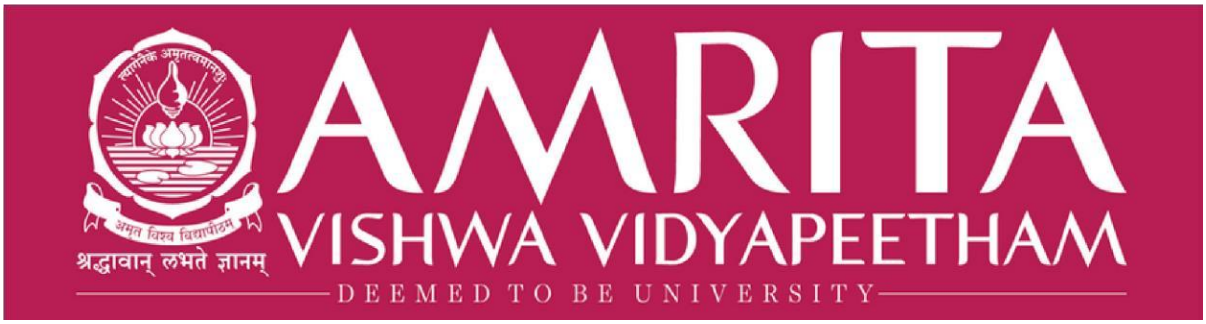


DATA STRUCTURES AND ALGORITHMS

23CSE203

Lab Manual

Submitted By

NAME	Akhilesh Mannam
ROLL NO	AV.SC.U4CSE25218
YEAR/SEM/SECTION	2nd YEAR/ SEM-3/CSE-C
CREDITS	05

Submitted To

NAME	Dr Gayathri Parasa
DESIGNATION	Asst. Professor (Sr. Gr)
DEPARTMENT	School of computing

INDEX

S.NO	TITLE	PAGE NO
1.	**WEEK 1** **1(A).** A rocket launch system displays a countdown before launch. Instead of using a loop, implement the countdown using recursion. Write a recursive python program to display a countdown from n to 1, followed by "Launch!". **1(B).** Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate P^n(power) where: P= principal growth factor, n= number of years. **1(C).** A HR department stores employee ID's in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list. **1(D).** In a logistics company, the number of possible ways to arrange n different parcels are calculated using the factorial formula (n!). Develop a recursive program to calculate the factorial of a given number. **1(E).** Write a recursive python program to print the first n terms of the Fibonacci series.	
2.	**WEEK 2** **2(A).** Write a python program to implement linear search. **2(B).** Write a python program to implement binary search with and without sorted array as an input.	
3.	**WEEK 3** **3(A).** Write a python program to implement bubble sort. **3(B).** Write a python program to implement insertion sort. **3(C).** Write a python program to implement selection sort.	
4.	**WEEK 4** **4(A).** Write a python program to implement quick sort. **4(B).** Write a python program to implement merge sort.	
5.	**WEEK 5** **5(A).** Write a python program to implement a Singly Linked List with the below operations. 1) Create Linked List 2) Insert Beginning 3) Insert End 4) Insert at Index 5) Delete by Value 6) Delete First Node 7) Delete Last Node 8) Count number of nodes 9) Sort the data of SLL 10) Search data in SLL 11) Display or Traverse 12) Exit	
6.	**WEEK 6** Write a python program to implement a Double Linked List with the below operations. 1) Create Linked List 2) Insert Beginning 3) Insert End 4) Insert at Index	

	5) Delete by Value 6) Delete First Node 7) Delete Last Node 8) Count number of nodes 9) Display or Traverse 10) Exit	
7.	**WEEK 7** **7(A).** Write a python program to implement stack operations such as push, pop, peek, display using an array. **7(B).** Write a python program to implement stack operations such as push, pop, peek, display using a linked list.	
8.	**WEEK 8** **8(A).** Write a python program to implement Queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach. Implement using array, and linked list data structures. **8(B).** Write a python program to implement Circular Queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach. Implement using array, and linked list data structures.	
9.	**WEEK 9** Write a python program to implement a Binary Search Tree (BST) and perform the following operations. 1. Insertion 2. Deletion 3. Tree Traversals – in-order, pre-order and post-order 4. Count Leaf Nodes 5. Count Internal Nodes 6. Height of the Tree	
10.	**WEEK 10** Write a menu driven python program to implement an AVL Tree and perform the following operations. 1. Insertion 2. Deletion 3. Tree Traversals 4. Count Leaf Nodes 5. Count Internal Nodes 6. Height of the Tree	
11.	**WEEK 11** Write a menu-driven Python program to implement a Binary Heap (Max Heap and Min Heap) and perform the following operations: 1. Insertion of an element into the Heap. 2. Deletion of the root element from the Heap. 3. Display the array representation of the Heap. 4. Exit the program.	
12.	**WEEK 12** Write a program to implement the following three collision resolution techniques in a Hash Table: 1. Separate Chaining 2. Linear Probing 3. Quadratic Probing	

WEEK – 1

PROGRAM-1(A):

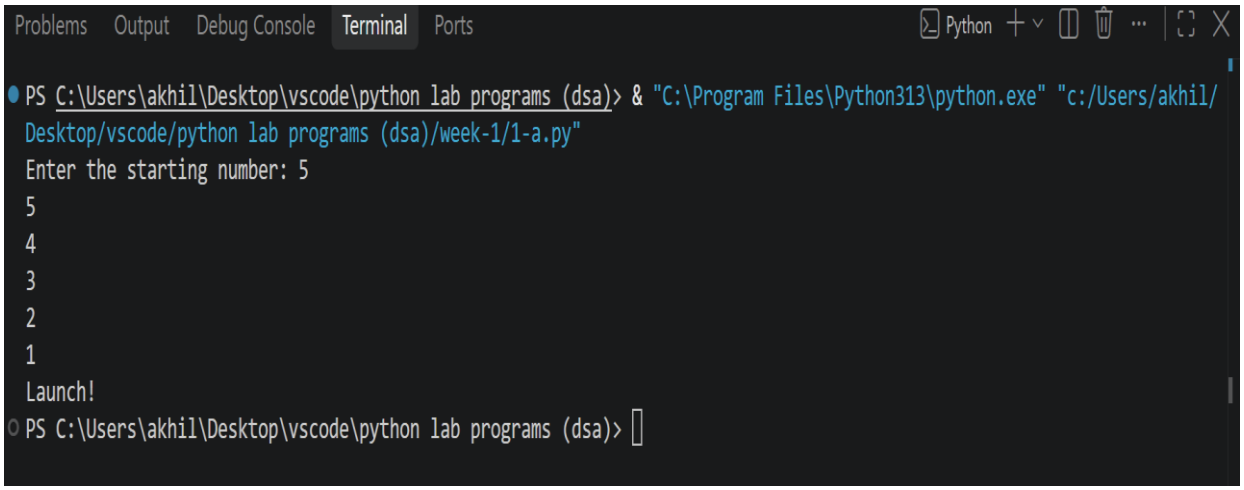
AIM: To write a recursive Python program to display a countdown from n to 1, followed by "Launch!".

CODE:

```
def countdown(n):
    if n == 0:
        print("Launch!")
        return
    print(n)
    countdown(n - 1)

n = int(input("Enter the starting number: "))
countdown(n)
```

OUTPUT:



PROGRAM-1(B):

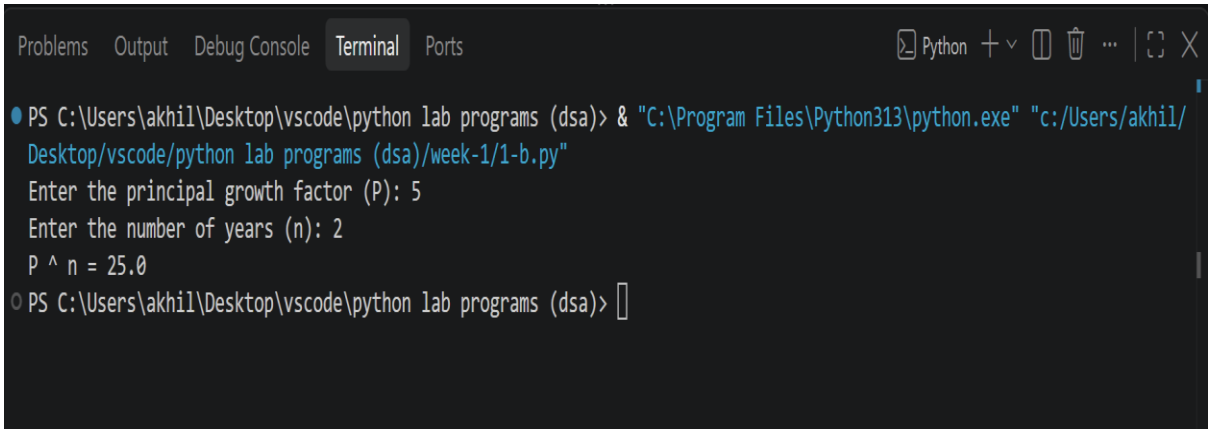
AIM: To write a recursive Python program to calculate P^n (P = principal growth factor, n = number of years) for compound growth of an investment.

CODE:

```
def power(P, n):
    if n == 0:
        return 1
    return P * power(P, n - 1)

P = float(input("Enter the principal growth factor (P): "))
n = int(input("Enter the number of years (n): "))
print("P ^ n =", power(P, n))
```

OUTPUT:



PROGRAM-1(C):

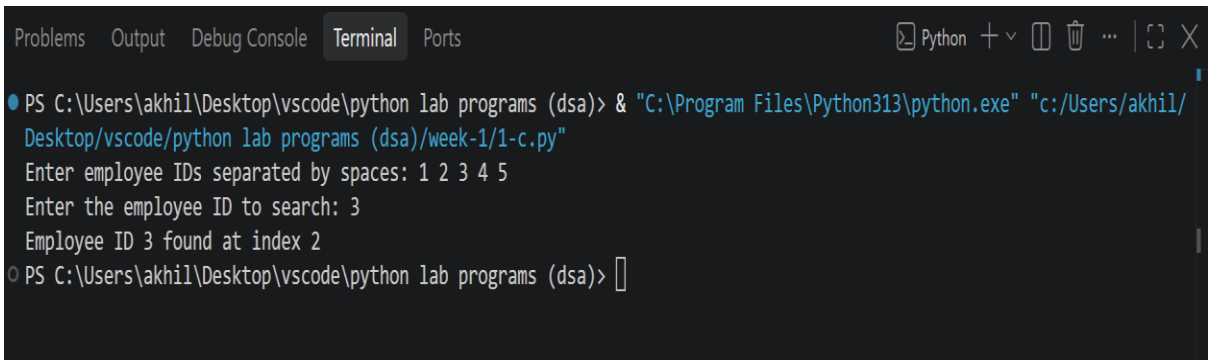
AIM: To write a recursive Python function to search for a given employee ID in a list.

CODE:

```
def search_id(ids, key, index=0):
    if index == len(ids):
        return -1
    if ids[index] == key:
        return index
    return search_id(ids, key, index + 1)

ids = list(map(int, input("Enter employee IDs separated by spaces:
").split()))
key = int(input("Enter the employee ID to search: "))
result = search_id(ids, key)
if result != -1:
    print("Employee ID", key, "found at index", result)
else:
    print("Employee ID not found")
```

OUTPUT:



PROGRAM-1(D):

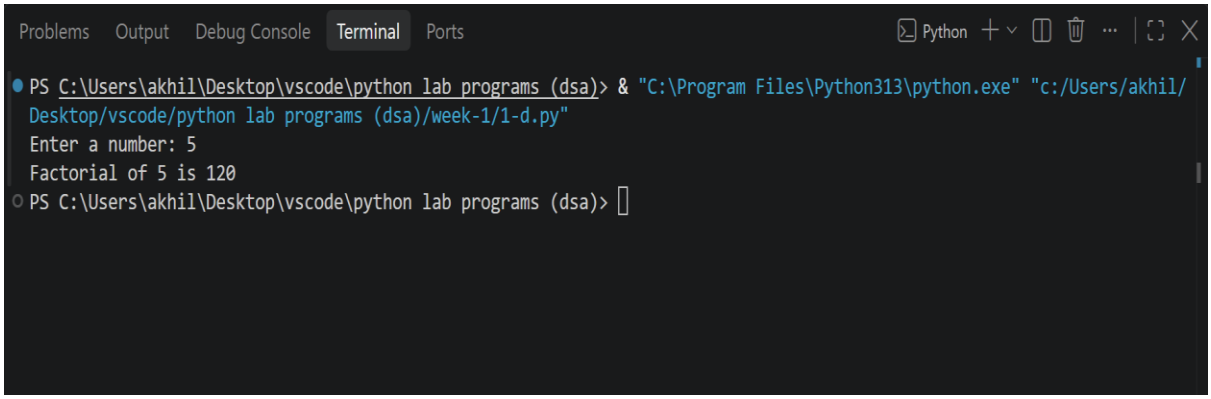
AIM: To write a recursive Python program to calculate the factorial of a given number.

CODE:

```python
def factorial(n):
    if n == 0 or n == 1:
        return 1
    return n * factorial(n - 1)

n = int(input("Enter a number: "))
if n < 0:
    print("Factorial is not defined for negative numbers")
else:
    print("Factorial of", n, "is", factorial(n))
```

OUTPUT:



PROGRAM-1(E):

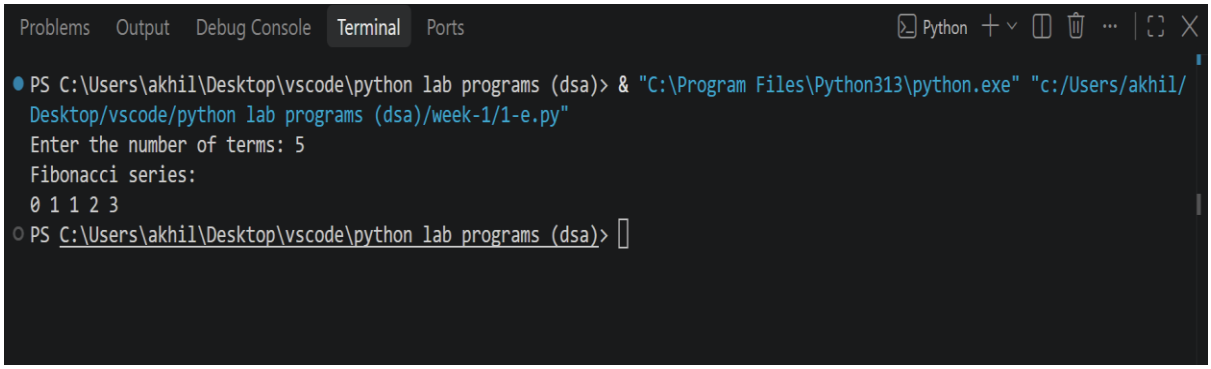
AIM: To write a recursive Python program to print the first n terms of the Fibonacci series.

CODE:

```python
def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)

n = int(input("Enter the number of terms: "))
print("Fibonacci series:")
for i in range(n):
    print(fibonacci(i), end=" ")
```

OUTPUT:



WEEK – 2

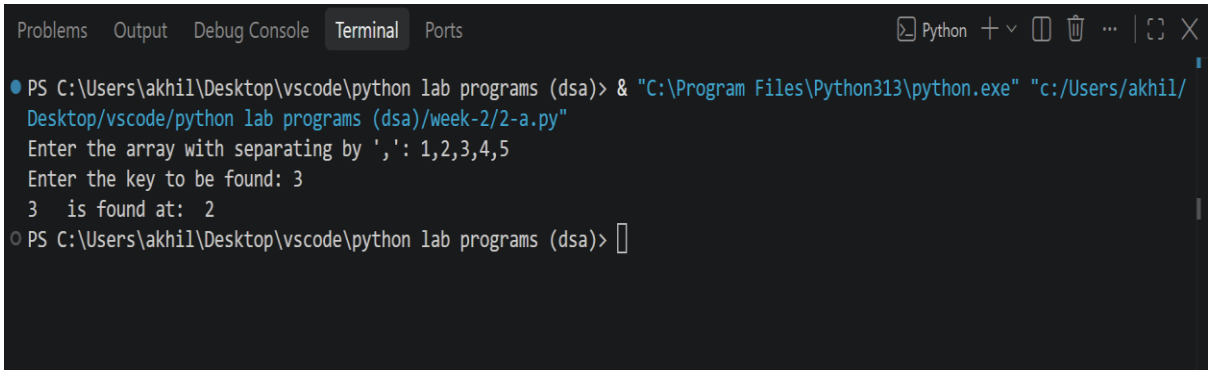
PROGRAM-2(A):

AIM: To write a Python program to implement linear search.

CODE:

```
def linear_search(arr,k):
    for i in range(len(arr)):
        if arr[i]==k:
            return i
    return -1
arr=list(map(int,input("Enter the array with separating by ',': ").split(',')))
k=int(input("Enter the key to be found: "))
re=linear_search(arr,k)
if re!=-1:
    print(k,"  is found at: ", re)
else:
    print("Not Found")
```

OUTPUT:



PROGRAM-2(B):

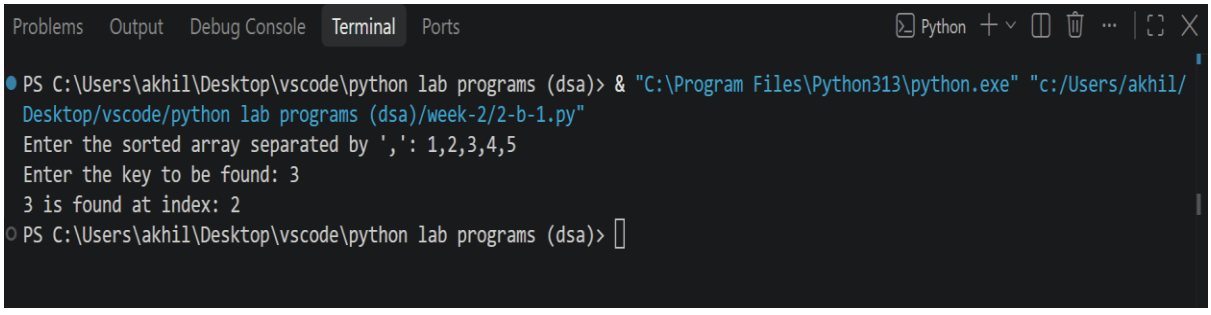
AIM: To write a Python program to implement binary search with and without a sorted array as an input.

CODE:

Binary Search – sorted array

```
def binary_search(arr, k):
    lo = 0
    high = len(arr) - 1
    while lo <= high:
        mid = (lo + high) // 2
        if arr[mid] == k:
            return mid
        elif arr[mid] > k:
            high = mid - 1
        else:
            lo = mid + 1
    return -1
arr = list(map(int, input("Enter the sorted array separated by ',':
").split(',')))
k = int(input("Enter the key to be found: "))
re = binary_search(arr, k)
if re != -1:
    print(k, "is found at index:", re)
else:
    print("Not Found")
```

OUTPUT:

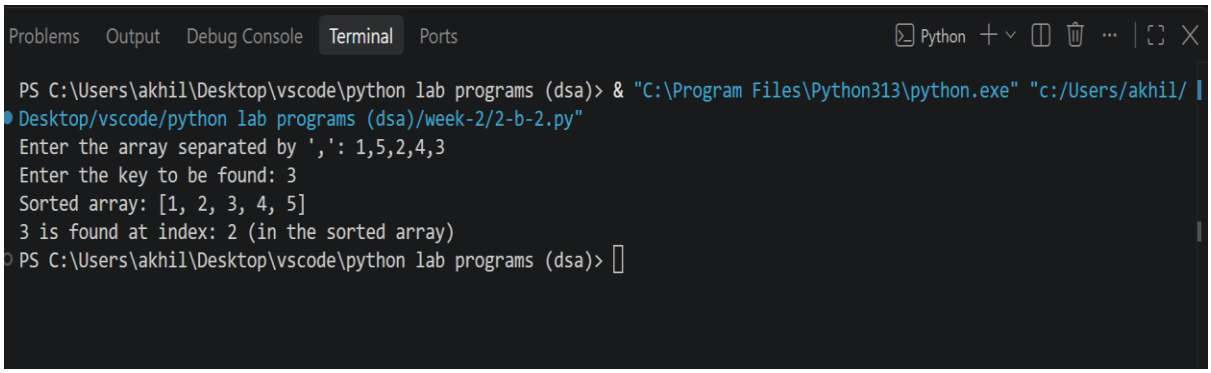


Binary Search – unsorted array

```
def selection_sort(arr):
    for i in range(len(arr) - 1):
        min_idx = i
        for j in range(i + 1, len(arr)):
            if arr[j] < arr[min_idx]:
                min_idx = j
```

```
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
def binary_search(arr, k):
    lo = 0
    high = len(arr) - 1
    while lo <= high:
        mid = (lo + high) // 2
        if arr[mid] == k:
            return mid
        elif arr[mid] > k:
            high = mid - 1
        else:
            lo = mid + 1
    return -1
arr = list(map(int, input("Enter the array separated by ',': ").split(',')))
k = int(input("Enter the key to be found: "))
selection_sort(arr)
print("Sorted array:", arr)
re = binary_search(arr, k)
if re != -1:
    print(k, "is found at index:", re, "(in the sorted array)")
else:
    print("Not Found")
```

OUTPUT:



WEEK – 3

PROGRAM-3(A):

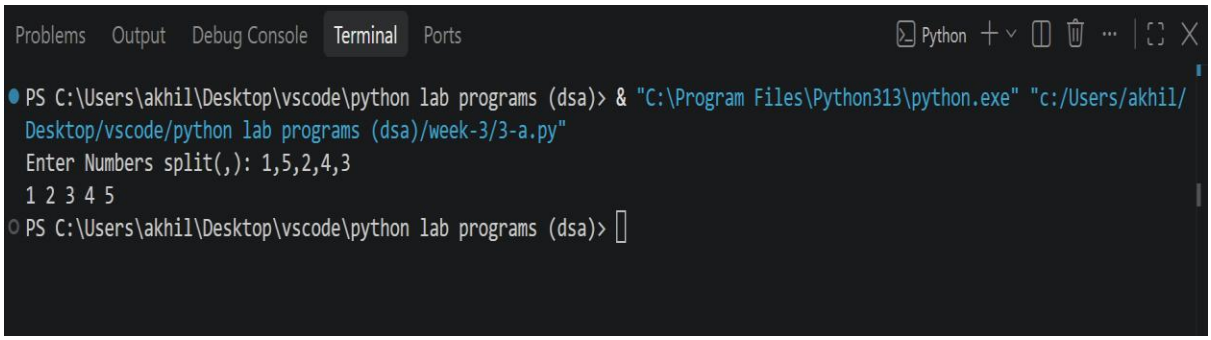
AIM: To write a Python program to implement bubble sort.

CODE:

```
def BubbleSort(arr):
    for i in range(len(arr)-1):
        for j in range(len(arr)-i-1):
            if arr[j]>arr[j+1]:
                arr[j],arr[j+1]=arr[j+1],arr[j]

arr=list(map(int,input("Enter Numbers split(,): ").split(',')))
BubbleSort(arr)
for i in arr:
    print(i,end=" ")
```

OUTPUT:



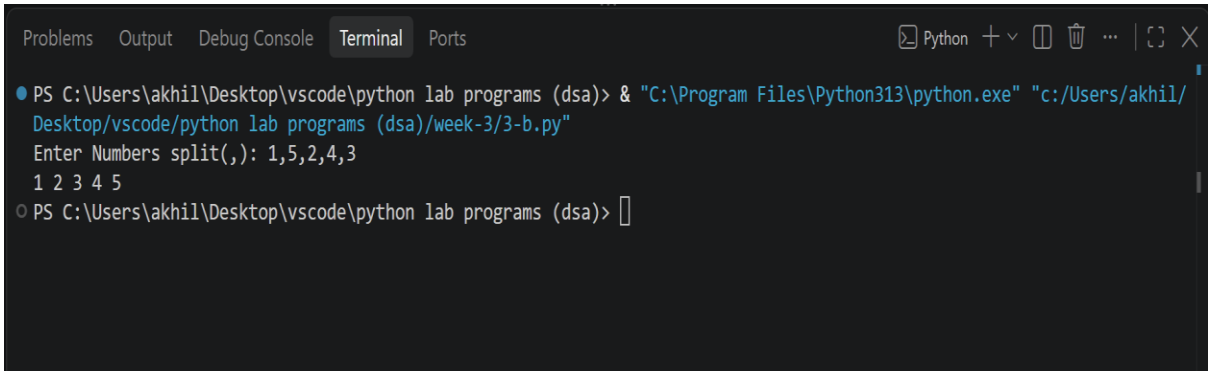
PROGRAM-3(B):

AIM: To write a Python program to implement insertion sort.

CODE:

```
def Insertion_Sort(arr):
    for i in range(1,len(arr)):
        key=arr[i]
        j=i-1
        while j>=0 and arr[j]>key:
            arr[j+1]=arr[j]
            j=j-1
        arr[j+1]=key
arr=list(map(int,input("Enter Numbers split(,): ").split(',')))
Insertion_Sort(arr)
for i in arr:
    print(i,end=" ")
```

OUTPUT:



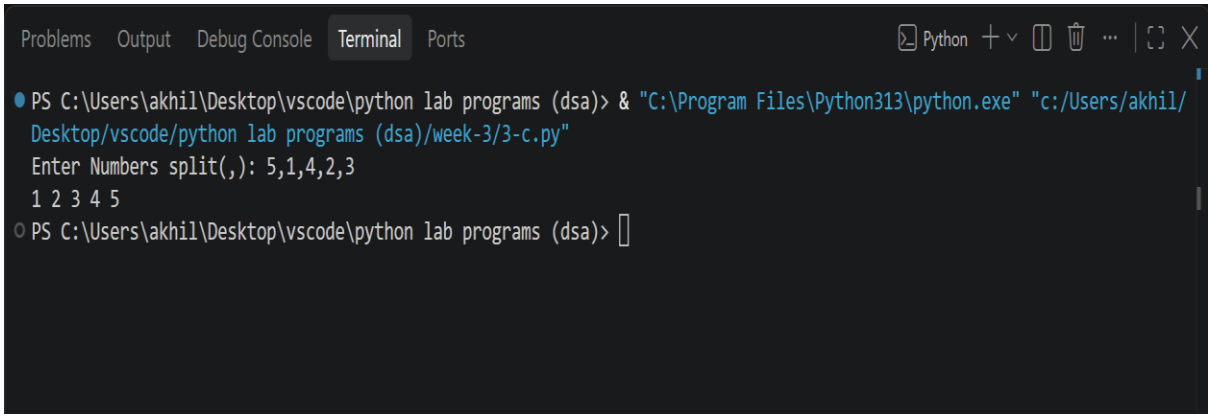
PROGRAM-3(C):

AIM: To write a Python program to implement selection sort.

CODE:

```
def Selection_Sort(arr):
    for i in range(len(arr)-1):
        min=i
        for j in range(i+1,len(arr)):
            if arr[j]<arr[min]:
                min=j
        arr[i],arr[min]=arr[min],arr[i]
arr=list(map(int,input("Enter Numbers split(,): ").split(',')))
Selection_Sort(arr)
for i in arr:
    print(i,end=" ")
```

OUTPUT:



WEEK – 4

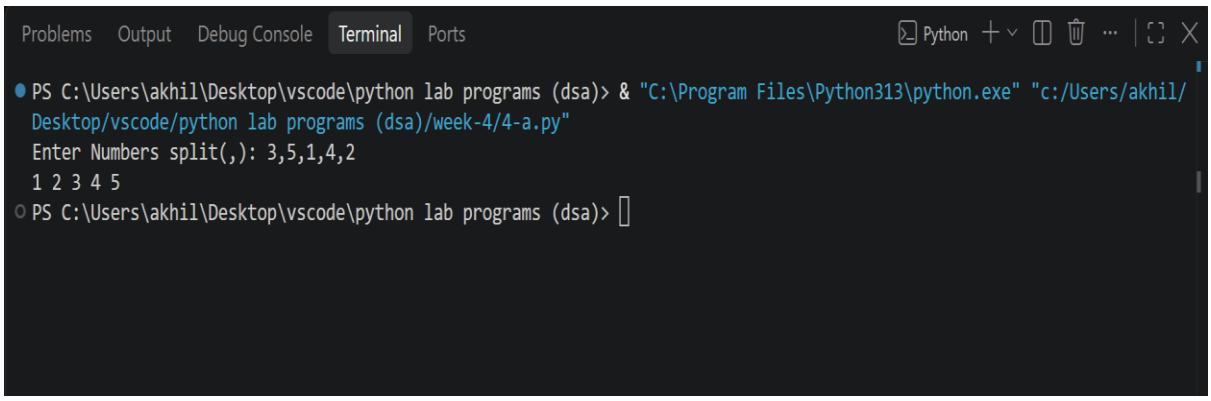
PROGRAM-4(A):

AIM: To write a Python program to implement quick sort.

CODE:

```
def quick_sort(arr, low, high):
    if low < high:
        pivot = arr[high]
        i = low - 1
        for j in range(low, high):
            if arr[j] <= pivot:
                i += 1
                arr[i], arr[j] = arr[j], arr[i]
        arr[i + 1], arr[high] = arr[high], arr[i + 1]
        p = i + 1
        quick_sort(arr, low, p - 1)
        quick_sort(arr, p + 1, high)
arr = list(map(int, input("Enter Numbers split(,): ").split(',')))
quick_sort(arr, 0, len(arr) - 1)
for i in arr:
    print(i, end=" ")
```

OUTPUT:



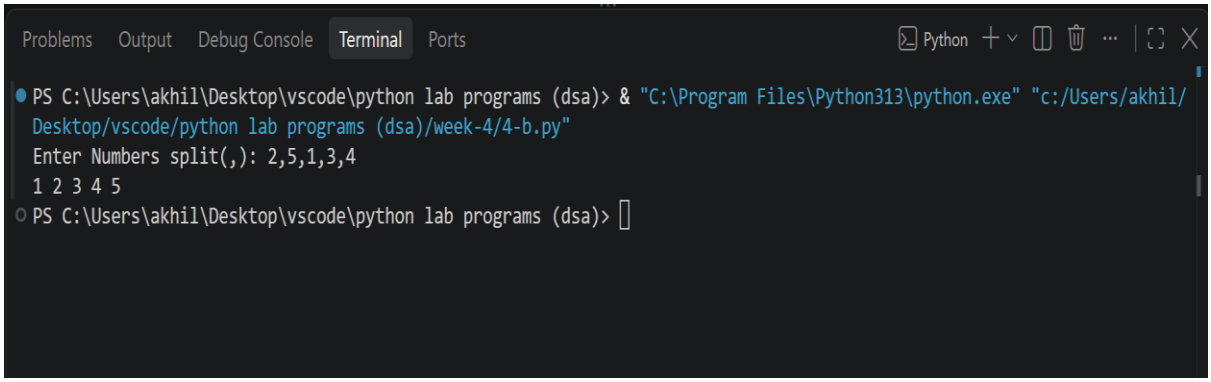
PROGRAM-4(B):

AIM: To write a Python program to implement merge sort.

CODE:

```
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2
        left = arr[:mid]
        right = arr[mid:]
        merge_sort(left)
        merge_sort(right)
        i = j = k = 0
        while i < len(left) and j < len(right):
            if left[i] <= right[j]:
                arr[k] = left[i]
                i += 1
            else:
                arr[k] = right[j]
                j += 1
            k += 1
        while i < len(left):
            arr[k] = left[i]
            i += 1
            k += 1
        while j < len(right):
            arr[k] = right[j]
            j += 1
            k += 1
arr = list(map(int, input("Enter Numbers split(,): ").split(',')))
merge_sort(arr)
for i in arr:
    print(i, end=" ")
```

OUTPUT:



WEEK – 5

PROGRAM-5:

AIM: To write a Python program to implement a Singly Linked List with the operations: Create, Insert at Beginning, Insert at End, Insert at Index, Delete by Value, Delete First Node, Delete Last Node, Count number of nodes, Sort, Search, Display/Traverse and Exit.

CODE:

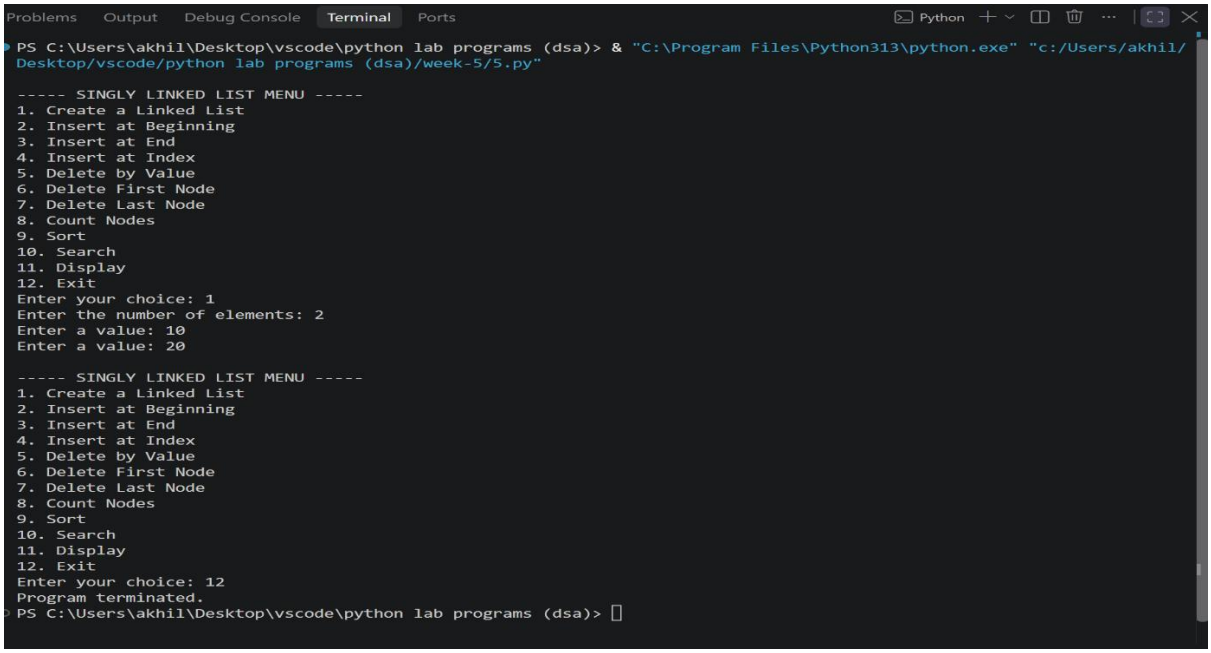
```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
class SinglyLinkedList:
    def __init__(self):
        self.head = None
    def create(self):
        n = int(input("Enter the number of elements: "))
        for i in range(n):
            x = int(input("Enter a value: "))
            self.insert_at_end(x)
    def display(self):
        if self.head is None:
            print("List is empty")
            return
        temp = self.head
        while temp:
            print(temp.data, end=" --> ")
            temp = temp.next
        print("None")
    def insert_at_beg(self, data):
        new = Node(data)
        new.next = self.head
        self.head = new
    def insert_at_end(self, data):
        new = Node(data)
        if self.head is None:
            self.head = new
            return
        temp = self.head
        while temp.next:
            temp = temp.next
        temp.next = new
    def insert_at_index(self, index, data):
        if index == 0:
            self.insert_at_beg(data)
            return
        temp = self.head
```

```
        for i in range(index - 1):
            if temp is None:
                print("Invalid Index")
                return
            temp = temp.next
        if temp is None:
            print("Invalid Index")
            return
        new = Node(data)
        new.next = temp.next
        temp.next = new
    def delete_first(self):
        if self.head is None:
            print("List is empty")
            return
        print("Deleted value is:", self.head.data)
        self.head = self.head.next
    def delete_last(self):
        if self.head is None:
            print("List is empty")
            return
        if self.head.next is None:
            print("Deleted value is:", self.head.data)
            self.head = None
            return
        temp = self.head
        while temp.next.next:
            temp = temp.next
        print("Deleted value is:", temp.next.data)
        temp.next = None
    def delete_by_value(self, value):
        if self.head is None:
            print("List is empty")
            return
        if self.head.data == value:
            self.head = self.head.next
            print("Value deleted")
            return
        prev = self.head
        temp = self.head.next
        while temp and temp.data != value:
            prev = temp
            temp = temp.next
        if temp is None:
            print("Value not found")
        else:
            prev.next = temp.next
            print("Value deleted")
```

```
    def count(self):
        c = 0
        temp = self.head
        while temp:
            c += 1
            temp = temp.next
        print("Number of nodes in the list:", c)
    def search(self, k):
        temp = self.head
        pos = 0
        while temp:
            if temp.data == k:
                print(k, "is found at position", pos)
                return
            temp = temp.next
            pos += 1
        print(k, "not found")
    def sort(self):
        if self.head is None:
            print("Cannot perform sorting")
            return
        i = self.head
        while i:
            j = i.next
            while j:
                if i.data > j.data:
                    i.data, j.data = j.data, i.data
                j = j.next
            i = i.next
        print("Sorting completed")
s = SinglyLinkedList()
while True:
    print("\n----- SINGLY LINKED LIST MENU -----")
    print("1. Create a Linked List")
    print("2. Insert at Beginning")
    print("3. Insert at End")
    print("4. Insert at Index")
    print("5. Delete by Value")
    print("6. Delete First Node")
    print("7. Delete Last Node")
    print("8. Count Nodes")
    print("9. Sort")
    print("10. Search")
    print("11. Display")
    print("12. Exit")
    choice = int(input("Enter your choice: "))
    match choice:
        case 1:
```

```
            s.create()
        case 2:
            s.insert_at_beg(int(input("Enter a value to insert at beginning:
")))
        case 3:
            s.insert_at_end(int(input("Enter a value to insert at end: ")))
        case 4:
            index = int(input("Enter the index to insert: "))
            data = int(input("Enter a value to insert: "))
            s.insert_at_index(index, data)
        case 5:
            s.delete_by_value(int(input("Enter the value to delete: ")))
        case 6:
            s.delete_first()
        case 7:
            s.delete_last()
        case 8:
            s.count()
        case 9:
            s.sort()
        case 10:
            s.search(int(input("Enter the value to search: ")))
        case 11:
            s.display()
        case 12:
            print("Program terminated.")
            break
        case _:
            print("Invalid choice")
```

OUTPUT:



WEEK – 6

PROGRAM-6:

AIM: To write a Python program to implement a Doubly Linked List with the operations: Create, Insert at Beginning, Insert at End, Insert at Index, Delete by Value, Delete First Node, Delete Last Node, Count number of nodes, Display/Traverse and Exit.

CODE:

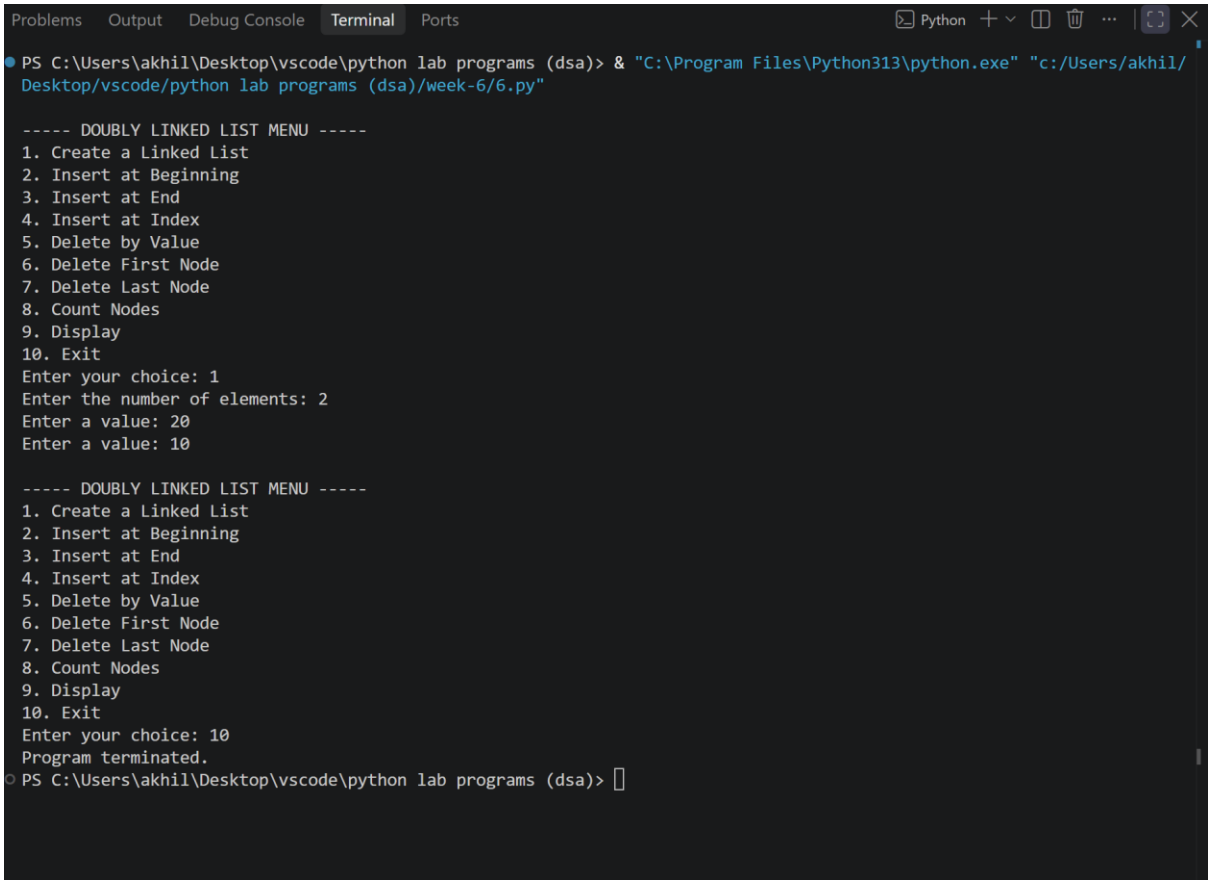
```
class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None
class DoublyLinkedList:
    def __init__(self):
        self.head = None
    def create(self):
        n = int(input("Enter the number of elements: "))
        for i in range(n):
            x = int(input("Enter a value: "))
            self.insert_at_end(x)
    def display(self):
        if self.head is None:
            print("List is empty")
            return
        temp = self.head
        print("None <-> ", end="")
        while temp:
            print(temp.data, end=" <-> ")
            temp = temp.next
        print("None")
    def insert_at_beg(self, data):
        new = Node(data)
        if self.head is not None:
            new.next = self.head
            self.head.prev = new
        self.head = new
    def insert_at_end(self, data):
        new = Node(data)
        if self.head is None:
            self.head = new
            return
        temp = self.head
        while temp.next:
            temp = temp.next
        temp.next = new
        new.prev = temp
```

```
    def insert_at_index(self, index, data):
        if index == 0:
            self.insert_at_beg(data)
            return
        temp = self.head
        for i in range(index - 1):
            if temp is None:
                print("Invalid Index")
                return
            temp = temp.next
        if temp is None:
            print("Invalid Index")
            return
        new = Node(data)
        new.next = temp.next
        new.prev = temp
        if temp.next:
            temp.next.prev = new
        temp.next = new
    def delete_first(self):
        if self.head is None:
            print("List is empty")
            return
        print("Deleted value is:", self.head.data)
        self.head = self.head.next
        if self.head:
            self.head.prev = None
    def delete_last(self):
        if self.head is None:
            print("List is empty")
            return
        temp = self.head
        while temp.next:
            temp = temp.next
        print("Deleted value is:", temp.data)
        if temp.prev:
            temp.prev.next = None
        else:
            self.head = None
    def delete_by_value(self, value):
        temp = self.head
        while temp and temp.data != value:
            temp = temp.next
        if temp is None:
            print("Value not found")
            return
        if temp.prev:
            temp.prev.next = temp.next
```

```
        else:
            self.head = temp.next
        if temp.next:
            temp.next.prev = temp.prev
        print("Value deleted")
    def count(self):
        c = 0
        temp = self.head
        while temp:
            c += 1
            temp = temp.next
        print("Number of nodes in the list:", c)
d = DoublyLinkedList()
while True:
    print("\n----- DOUBLY LINKED LIST MENU -----")
    print("1. Create a Linked List")
    print("2. Insert at Beginning")
    print("3. Insert at End")
    print("4. Insert at Index")
    print("5. Delete by Value")
    print("6. Delete First Node")
    print("7. Delete Last Node")
    print("8. Count Nodes")
    print("9. Display")
    print("10. Exit")
    choice = int(input("Enter your choice: "))
    match choice:
        case 1:
            d.create()
        case 2:
            d.insert_at_beg(int(input("Enter a value to insert at beginning:
")))
        case 3:
            d.insert_at_end(int(input("Enter a value to insert at end: ")))
        case 4:
            index = int(input("Enter the index to insert: "))
            data = int(input("Enter a value to insert: "))
            d.insert_at_index(index, data)
        case 5:
            d.delete_by_value(int(input("Enter the value to delete: ")))
        case 6:
            d.delete_first()
        case 7:
            d.delete_last()
        case 8:
            d.count()
        case 9:
            d.display()
```

```
        case 10:
            print("Program terminated.")
            break
        case _:
            print("Invalid choice")
```

OUTPUT:



WEEK – 7

PROGRAM-7(A):

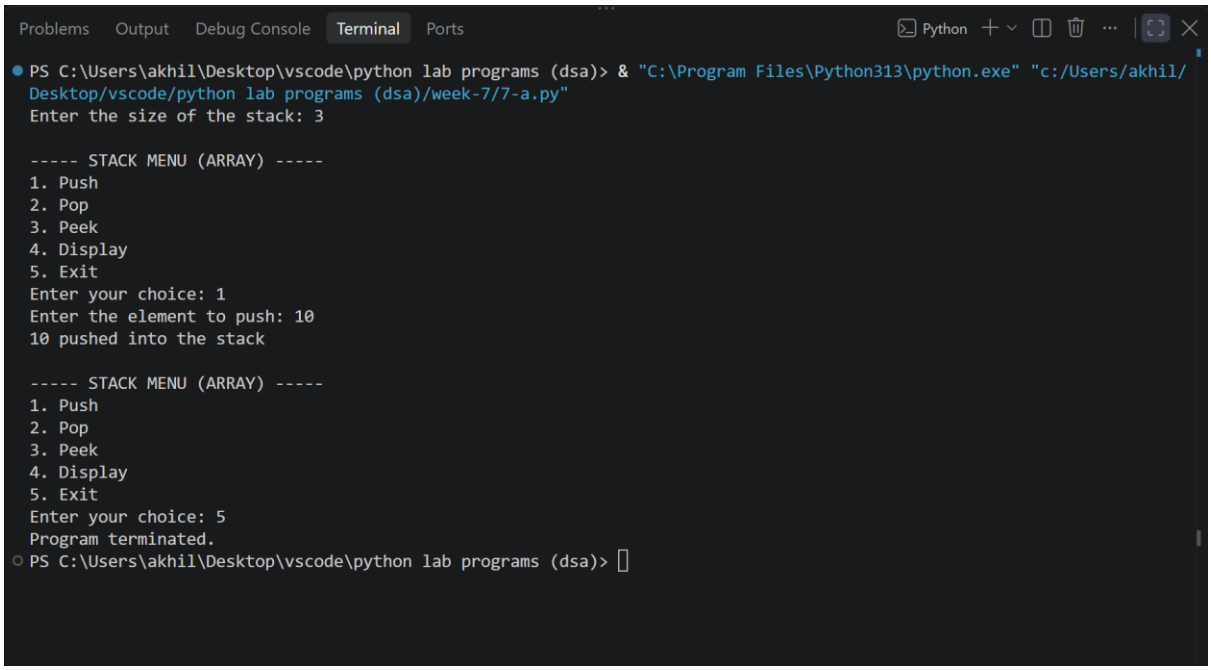
AIM: To write a Python program to implement stack operations such as push, pop, peek and display using an array.

CODE:

```
class Stack:
    def __init__(self, size):
        self.size = size
        self.stack = [None] * size
        self.top = -1
    def push(self, item):
        if self.top == self.size - 1:
            print("Stack Overflow")
        else:
            self.top += 1
            self.stack[self.top] = item
            print(item, "pushed into the stack")
    def pop(self):
        if self.top == -1:
            print("Stack Underflow")
        else:
            print(self.stack[self.top], "popped from the stack")
            self.stack[self.top] = None
            self.top -= 1
    def peek(self):
        if self.top == -1:
            print("Stack is empty")
        else:
            print("Top element is:", self.stack[self.top])
    def display(self):
        if self.top == -1:
            print("Stack is empty")
        else:
            print("The elements of the stack (top to bottom) are:")
            for i in range(self.top, -1, -1):
                print(self.stack[i])
size = int(input("Enter the size of the stack: "))
s = Stack(size)
while True:
    print("\n----- STACK MENU (ARRAY) -----")
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")
```

```
    choice = int(input("Enter your choice: "))
    if choice == 1:
        s.push(int(input("Enter the element to push: ")))
    elif choice == 2:
        s.pop()
    elif choice == 3:
        s.peek()
    elif choice == 4:
        s.display()
    elif choice == 5:
        print("Program terminated.")
        break
    else:
        print("Invalid choice")
```

OUTPUT:



PROGRAM-7(B):

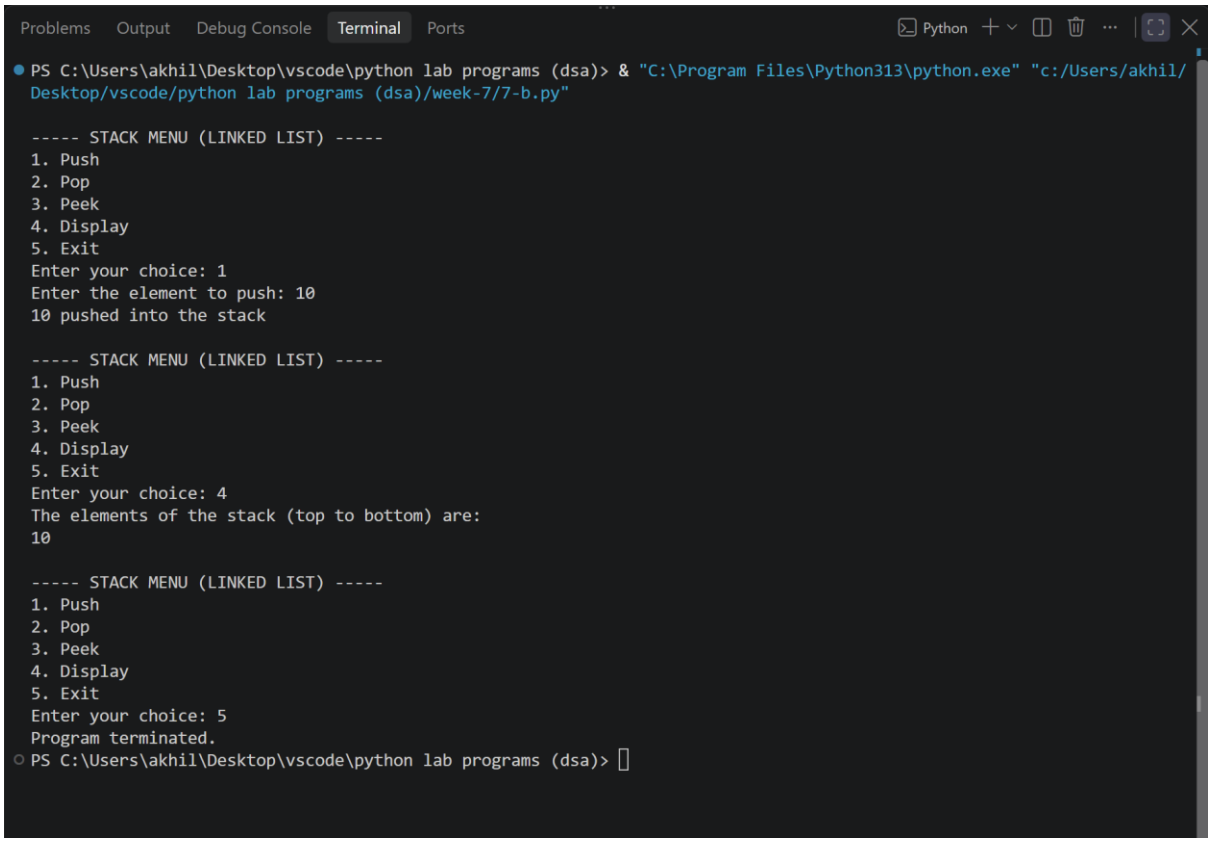
AIM: To write a Python program to implement stack operations such as push, pop, peek and display using a linked list.

CODE:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
class Stack:
    def __init__(self):
        self.top = None
    def push(self, item):
        new = Node(item)
        new.next = self.top
        self.top = new
        print(item, "pushed into the stack")
    def pop(self):
        if self.top is None:
            print("Stack Underflow")
        else:
            print(self.top.data, "popped from the stack")
            self.top = self.top.next
    def peek(self):
        if self.top is None:
            print("Stack is empty")
        else:
            print("Top element is:", self.top.data)
    def display(self):
        if self.top is None:
            print("Stack is empty")
        else:
            print("The elements of the stack (top to bottom) are:")
            temp = self.top
            while temp:
                print(temp.data)
                temp = temp.next
s = Stack()
while True:
    print("\n----- STACK MENU (LINKED LIST) -----")
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")
    choice = int(input("Enter your choice: "))
    if choice == 1:
```

```
        s.push(int(input("Enter the element to push: ")))
    elif choice == 2:
        s.pop()
    elif choice == 3:
        s.peek()
    elif choice == 4:
        s.display()
    elif choice == 5:
        print("Program terminated.")
        break
    else:
        print("Invalid choice")
```

OUTPUT:



WEEK – 8

PROGRAM-8(A):

AIM: To write a Python program to implement a Queue and its operations such as enqueue, dequeue, peek and display with a menu driven approach, using array and linked list data structures.

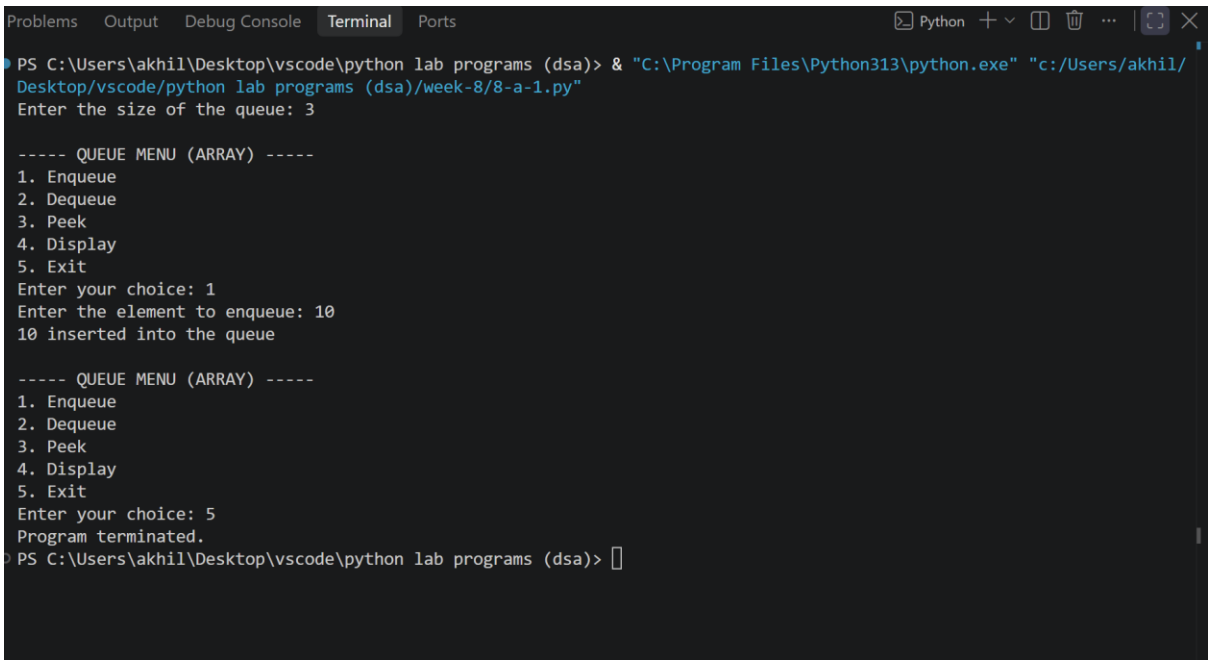
CODE:

Queue using array

```
class Queue:
    def __init__(self, size):
        self.size = size
        self.queue = [None] * size
        self.front = -1
        self.rear = -1
    def enqueue(self, item):
        if self.rear == self.size - 1:
            print("Queue Overflow")
        else:
            if self.front == -1:
                self.front = 0
            self.rear += 1
            self.queue[self.rear] = item
            print(item, "inserted into the queue")
    def dequeue(self):
        if self.front == -1:
            print("Queue Underflow")
        else:
            print(self.queue[self.front], "deleted from the queue")
            self.queue[self.front] = None
            self.front += 1
            if self.front > self.rear:
                self.front = -1
                self.rear = -1
    def peek(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("Front element is:", self.queue[self.front])
    def display(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("The elements of the queue are:")
            for i in range(self.front, self.rear + 1):
                print(self.queue[i])
size = int(input("Enter the size of the queue: "))
```

```python
q = Queue(size)
while True:
    print("\n----- QUEUE MENU (ARRAY) -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")
    choice = int(input("Enter your choice: "))
    if choice == 1:
        q.enqueue(int(input("Enter the element to enqueue: ")))
    elif choice == 2:
        q.dequeue()
    elif choice == 3:
        q.peek()
    elif choice == 4:
        q.display()
    elif choice == 5:
        print("Program terminated.")
        break
    else:
        print("Invalid choice")
```

OUTPUT:



Queue using linked list

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
class Queue:
    def __init__(self):
        self.front = None
        self.rear = None
    def enqueue(self, item):
        new = Node(item)
        if self.rear is None:
            self.front = new
            self.rear = new
        else:
            self.rear.next = new
            self.rear = new
        print(item, "inserted into the queue")
    def dequeue(self):
        if self.front is None:
            print("Queue Underflow")
        else:
            print(self.front.data, "deleted from the queue")
            self.front = self.front.next
            if self.front is None:
                self.rear = None
    def peek(self):
        if self.front is None:
            print("Queue is empty")
        else:
            print("Front element is:", self.front.data)
    def display(self):
        if self.front is None:
            print("Queue is empty")
        else:
            print("The elements of the queue are:")
            temp = self.front
            while temp:
                print(temp.data)
                temp = temp.next
q = Queue()
while True:
    print("\n----- QUEUE MENU (LINKED LIST) -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
```

```
    print("5. Exit")
    choice = int(input("Enter your choice: "))
    if choice == 1:
        q.enqueue(int(input("Enter the element to enqueue: ")))
    elif choice == 2:
        q.dequeue()
    elif choice == 3:
        q.peek()
    elif choice == 4:
        q.display()
    elif choice == 5:
        print("Program terminated.")
        break
    else:
        print("Invalid choice")
```

OUTPUT:



PROGRAM-8(B):

AIM: To write a Python program to implement a Circular Queue and its operations such as enqueue, dequeue, peek and display with a menu driven approach, using array and linked list data structures.

CODE:

Circular Queue using array

```
class CircularQueue:
    def __init__(self, size):
        self.size = size
        self.queue = [None] * size
        self.front = -1
        self.rear = -1
    def enqueue(self, item):
        if (self.rear + 1) % self.size == self.front:
            print("Queue Overflow")
        elif self.front == -1:
            self.front = 0
            self.rear = 0
            self.queue[self.rear] = item
            print(item, "inserted into the queue")
        else:
            self.rear = (self.rear + 1) % self.size
            self.queue[self.rear] = item
            print(item, "inserted into the queue")
    def dequeue(self):
        if self.front == -1:
            print("Queue Underflow")
        elif self.front == self.rear:
            print(self.queue[self.front], "deleted from the queue")
            self.queue[self.front] = None
            self.front = -1
            self.rear = -1
        else:
            print(self.queue[self.front], "deleted from the queue")
            self.queue[self.front] = None
            self.front = (self.front + 1) % self.size
    def peek(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("Front element is:", self.queue[self.front])
    def display(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("The elements of the queue are:")
```

```
            i = self.front
            while True:
                print(self.queue[i])
                if i == self.rear:
                    break
                i = (i + 1) % self.size
size = int(input("Enter the size of the queue: "))
q = CircularQueue(size)
while True:
    print("\n----- CIRCULAR QUEUE MENU (ARRAY) -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")
    choice = int(input("Enter your choice: "))
    if choice == 1:
        q.enqueue(int(input("Enter the element to enqueue: ")))
    elif choice == 2:
        q.dequeue()
    elif choice == 3:
        q.peek()
    elif choice == 4:
        q.display()
    elif choice == 5:
        print("Program terminated.")
        break
    else:
        print("Invalid choice")
```

OUTPUT:

```
Problems  Output  Debug Console  Terminal  Ports                                   Python
PS C:\Users\akhil\Desktop\vscode\python lab programs (dsa)> & "C:\Program Files\Python313\python.exe" "c:/Users/akhil/
Desktop/vscode/python lab programs (dsa)/week-8/8-b-1.py"
Enter the size of the queue: 5

----- CIRCULAR QUEUE MENU (ARRAY) -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 10
10 inserted into the queue

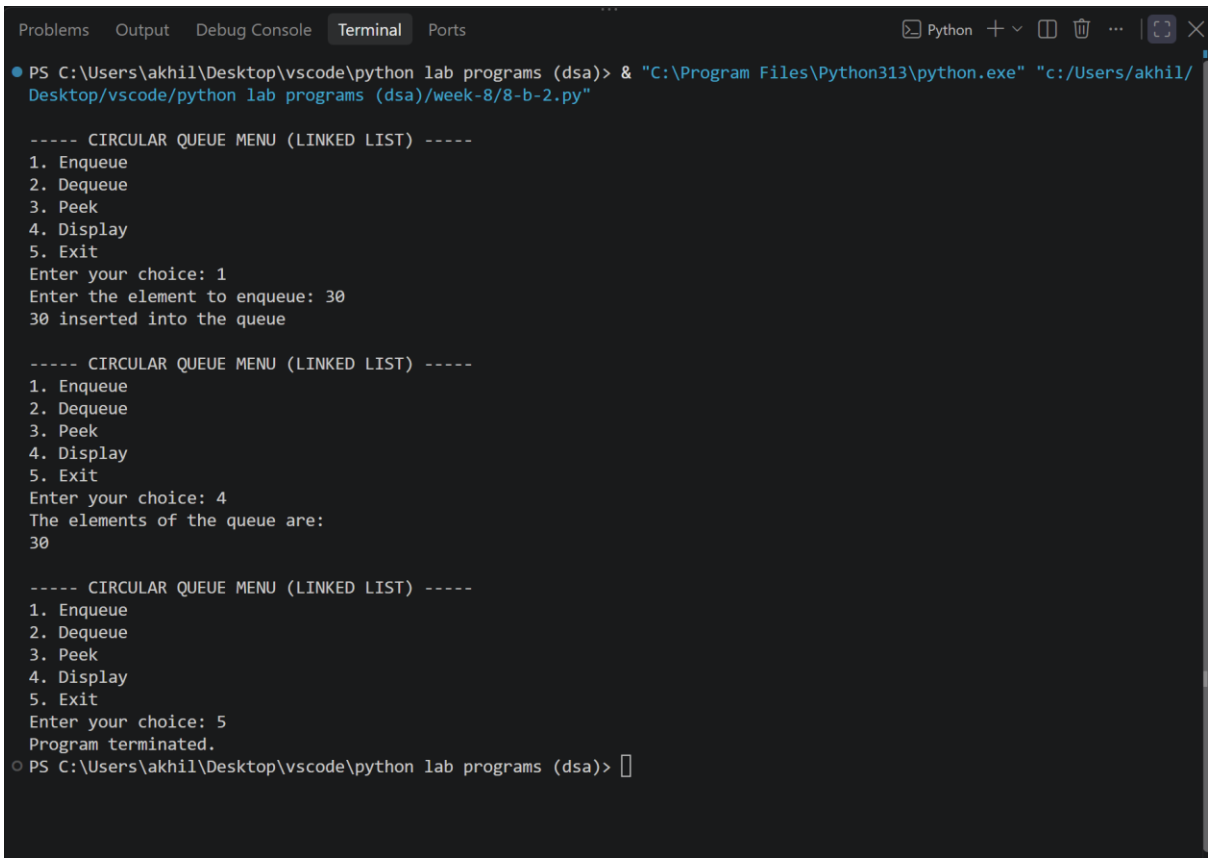
----- CIRCULAR QUEUE MENU (ARRAY) -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 5
Program terminated.
PS C:\Users\akhil\Desktop\vscode\python lab programs (dsa)>
```

Circular Queue using linked list

```python
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
class CircularQueue:
    def __init__(self):
        self.rear = None
    def enqueue(self, item):
        new = Node(item)
        if self.rear is None:
            new.next = new
        else:
            new.next = self.rear.next
            self.rear.next = new
        self.rear = new
        print(item, "inserted into the queue")
    def dequeue(self):
        if self.rear is None:
            print("Queue Underflow")
        else:
            front = self.rear.next
            print(front.data, "deleted from the queue")
            if front == self.rear:
                self.rear = None
            else:
                self.rear.next = front.next
    def peek(self):
        if self.rear is None:
            print("Queue is empty")
        else:
            print("Front element is:", self.rear.next.data)
    def display(self):
        if self.rear is None:
            print("Queue is empty")
        else:
            print("The elements of the queue are:")
            temp = self.rear.next
            while True:
                print(temp.data)
                if temp == self.rear:
                    break
                temp = temp.next
q = CircularQueue()
while True:
    print("\n----- CIRCULAR QUEUE MENU (LINKED LIST) -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
```

```
    print("4. Display")
    print("5. Exit")
    choice = int(input("Enter your choice: "))
    if choice == 1:
        q.enqueue(int(input("Enter the element to enqueue: ")))
    elif choice == 2:
        q.dequeue()
    elif choice == 3:
        q.peek()
    elif choice == 4:
        q.display()
    elif choice == 5:
        print("Program terminated.")
        break
    else:
        print("Invalid choice")
```

OUTPUT:



WEEK – 9

PROGRAM-9:

AIM: To write a Python program to implement a Binary Search Tree (BST) and perform Insertion, Deletion, Tree Traversals (in-order, pre-order, post-order), Count Leaf Nodes, Count Internal Nodes and find the Height of the tree.

CODE:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None
def insert(root, x):
    if root is None:
        return Node(x)
    if x < root.data:
        root.left = insert(root.left, x)
    elif x > root.data:
        root.right = insert(root.right, x)
    else:
        print("Duplicate value is not allowed.")
    return root
def inorder(root):
    if root is None:
        return
    inorder(root.left)
    print(root.data, end=" ")
    inorder(root.right)
def preorder(root):
    if root is None:
        return
    print(root.data, end=" ")
    preorder(root.left)
    preorder(root.right)
def postorder(root):
    if root is None:
        return
    postorder(root.left)
    postorder(root.right)
    print(root.data, end=" ")
def find_min(root):
    if root is None:
        return None
    while root.left is not None:
        root = root.left
    return root
```

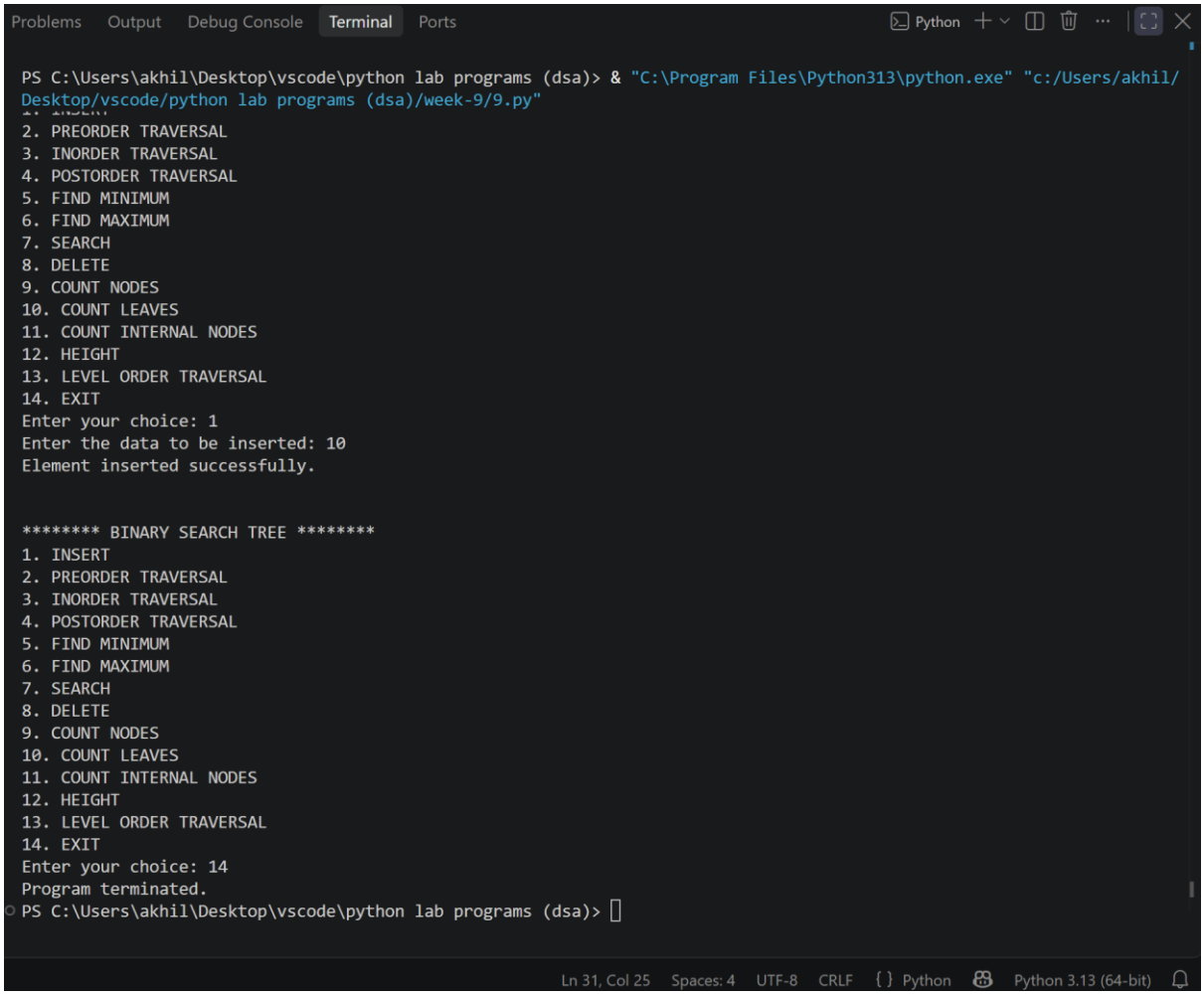
```
def find_max(root):
    if root is None:
        return None
    while root.right is not None:
        root = root.right
    return root
def search(root, data):
    if root is None:
        return None
    if data < root.data:
        return search(root.left, data)
    elif data > root.data:
        return search(root.right, data)
    else:
        return root
def count_nodes(root):
    if root is None:
        return 0
    return count_nodes(root.left) + count_nodes(root.right) + 1
def count_leaves(root):
    if root is None:
        return 0
    if root.left is None and root.right is None:
        return 1
    return count_leaves(root.left) + count_leaves(root.right)
def count_internal(root):
    if root is None:
        return 0
    if root.left is None and root.right is None:
        return 0
    return count_internal(root.left) + count_internal(root.right) + 1
def height(root):
    if root is None:
        return -1
    left_height = height(root.left)
    right_height = height(root.right)
    return max(left_height, right_height) + 1
def delete(root, x):
    if root is None:
        print("Element not found.")
        return None
    if x < root.data:
        root.left = delete(root.left, x)
    elif x > root.data:
        root.right = delete(root.right, x)
    else:
        if root.left is None and root.right is None:
            root = None
```

```
        elif root.left is None:
            root = root.right
        elif root.right is None:
            root = root.left
        else:
            successor = find_min(root.right)
            root.data = successor.data
            root.right = delete(root.right, successor.data)
    return root
def print_level(root, level):
    if root is None:
        return
    if level == 0:
        print(root.data, end=" ")
    else:
        print_level(root.left, level - 1)
        print_level(root.right, level - 1)
def level_order(root):
    h = height(root)
    for level in range(h + 1):
        print_level(root, level)
root = None
while True:
    print("\n\n******** BINARY SEARCH TREE ********")
    print("1. INSERT")
    print("2. PREORDER TRAVERSAL")
    print("3. INORDER TRAVERSAL")
    print("4. POSTORDER TRAVERSAL")
    print("5. FIND MINIMUM")
    print("6. FIND MAXIMUM")
    print("7. SEARCH")
    print("8. DELETE")
    print("9. COUNT NODES")
    print("10. COUNT LEAVES")
    print("11. COUNT INTERNAL NODES")
    print("12. HEIGHT")
    print("13. LEVEL ORDER TRAVERSAL")
    print("14. EXIT")
    choice = int(input("Enter your choice: "))
    if choice == 1:
        data = int(input("Enter the data to be inserted: "))
        root = insert(root, data)
        print("Element inserted successfully.")
    elif choice == 2:
        print("Preorder Traversal:", end=" ")
        if root is None:
            print("Tree is empty.")
        else:
```

```
            preorder(root)
            print()
    elif choice == 3:
        print("Inorder Traversal:", end=" ")
        if root is None:
            print("Tree is empty.")
        else:
            inorder(root)
            print()
    elif choice == 4:
        print("Postorder Traversal:", end=" ")
        if root is None:
            print("Tree is empty.")
        else:
            postorder(root)
            print()
    elif choice == 5:
        node = find_min(root)
        if node is None:
            print("Tree is empty.")
        else:
            print("Minimum element:", node.data)
    elif choice == 6:
        node = find_max(root)
        if node is None:
            print("Tree is empty.")
        else:
            print("Maximum element:", node.data)
    elif choice == 7:
        data = int(input("Enter the data to be searched: "))
        node = search(root, data)
        if node is None:
            print("Element", data, "is not found.")
        else:
            print("Element", data, "is found.")
    elif choice == 8:
        data = int(input("Enter the data to be deleted: "))
        root = delete(root, data)
    elif choice == 9:
        print("Number of nodes:", count_nodes(root))
    elif choice == 10:
        print("Number of leaf nodes:", count_leaves(root))
    elif choice == 11:
        print("Number of internal nodes:", count_internal(root))
    elif choice == 12:
        print("Height of BST:", height(root))
    elif choice == 13:
        print("Level Order Traversal:", end=" ")
```

```
        if root is None:
            print("Tree is empty.")
        else:
            level_order(root)
            print()
    elif choice == 14:
        print("Program terminated.")
        break
    else:
        print("Invalid choice. Please try again.")
```

OUTPUT:



WEEK – 10

PROGRAM-10:

AIM: To write a menu driven Python program to implement an AVL Tree and perform Insertion, Deletion, Tree Traversals, Count Leaf Nodes, Count Internal Nodes and find the Height of the tree.

CODE:

```
class Node:
    def __init__(self, x):
        self.left = None
        self.data = x
        self.right = None
        self.h = 0
def height(root):
    if root is None:
        return -1
    else:
        return root.h
def LL(x):
    y = x.left
    x.left = y.right
    y.right = x
    x.h = max(height(x.left), height(x.right)) + 1
    y.h = max(height(y.left), height(y.right)) + 1
    return y
def RR(x):
    y = x.right
    x.right = y.left
    y.left = x
    x.h = max(height(x.left), height(x.right)) + 1
    y.h = max(height(y.left), height(y.right)) + 1
    return y
def LR(k3):
    k3.left = RR(k3.left)
    return LL(k3)
def RL(k3):
    k3.right = LL(k3.right)
    return RR(k3)
def inorder(root):
    if root is not None:
        inorder(root.left)
        print(root.data, end=" ")
        inorder(root.right)
def preorder(root):
    if root is not None:
        print(root.data, end=" ")
```

```
        preorder(root.left)
        preorder(root.right)
def postorder(root):
    if root is not None:
        postorder(root.left)
        postorder(root.right)
        print(root.data, end=" ")
def count_leaves(root):
    if root is None:
        return 0
    if root.left is None and root.right is None:
        return 1
    return count_leaves(root.left) + count_leaves(root.right)
def count_internal(root):
    if root is None:
        return 0
    if root.left is None and root.right is None:
        return 0
    return count_internal(root.left) + count_internal(root.right) + 1
def insert(root, x):
    if root is None:
        return Node(x)
    if x < root.data:
        root.left = insert(root.left, x)
        if height(root.left) - height(root.right) == 2:
            if x < root.left.data:
                root = LL(root)
            else:
                root = LR(root)
    elif x > root.data:
        root.right = insert(root.right, x)
        if height(root.right) - height(root.left) == 2:
            if x > root.right.data:
                root = RR(root)
            else:
                root = RL(root)
    else:
        print("Duplicates not allowed")
    root.h = max(height(root.left),height(root.right)) + 1
    return root
def delete(root, x):
    if root is None:
        return None
    if x < root.data:
        root.left = delete(root.left, x)
        if height(root.left) - height(root.right) == 2:
            if x < root.left.data:
                root = LL(root)
```

```
            else:
                root = LR(root)
    elif x > root.data:
        root.right = delete(root.right, x)
        if height(root.right) - height(root.left) == 2:
            if x > root.right.data:
                root = RR(root)
            else:
                root = RL(root)
    else:
        if root.left is None and root.right is None:
            root = None
        elif root.right is None:
            root = root.left
        elif root.left is None:
            root = root.right
        else:
            successor = root.right
            while successor.left is not None:
                successor = successor.left
            root.data = successor.data
            root.right = delete(root.right, successor.data)
        root.h = max(height(root.left), height(root.right)) + 1
    return root
root = None
while True:
    print("\n***** AVL TREE MENU *****")
    print("1. INSERT")
    print("2. PREORDER TRAVERSAL")
    print("3. INORDER TRAVERSAL")
    print("4. POSTORDER TRAVERSAL")
    print("5. DELETE")
    print("6. COUNT LEAF NODES")
    print("7. COUNT INTERNAL NODES")
    print("8. HEIGHT OF THE TREE")
    print("9. EXIT")
    choice = int(input("Enter your choice: "))
    if choice == 1:
        data = int(input("Enter the data to be inserted: "))
        root = insert(root, data)
    elif choice == 2:
        print("Preorder Traversal:")
        preorder(root)
        print()
    elif choice == 3:
        print("Inorder Traversal:")
        inorder(root)
        print()
```

```
    elif choice == 4:
        print("Postorder Traversal:")
        postorder(root)
        print()
    elif choice == 5:
        data = int(input("Enter the data to be deleted: "))
        root = delete(root, data)
    elif choice == 6:
        print("Number of leaf nodes:", count_leaves(root))
    elif choice == 7:
        print("Number of internal nodes:", count_internal(root))
    elif choice == 8:
        print("Height of AVL tree:", height(root))
    elif choice == 9:
        print("Program terminated.")
        break
    else:
        print("Invalid choice!")
```

OUTPUT:

```
Problems  Output  Debug Console  Terminal  Ports                                    Python

PS C:\Users\akhil\Desktop\vscode\python lab programs (dsa)> & "C:\Program Files\Python313\python.exe" "c:/Users/akhil/
Desktop/vscode/python lab programs (dsa)/week-10/10.py"
3. INORDER TRAVERSAL
4. POSTORDER TRAVERSAL
5. DELETE
6. COUNT LEAF NODES
7. COUNT INTERNAL NODES
8. HEIGHT OF THE TREE
9. EXIT
Enter your choice: 1
Enter the data to be inserted: 30

***** AVL TREE MENU *****
1. INSERT
2. PREORDER TRAVERSAL
3. INORDER TRAVERSAL
4. POSTORDER TRAVERSAL
5. DELETE
6. COUNT LEAF NODES
7. COUNT INTERNAL NODES
8. HEIGHT OF THE TREE
9. EXIT
Enter your choice: 6
Number of leaf nodes: 1

***** AVL TREE MENU *****
1. INSERT
2. PREORDER TRAVERSAL
3. INORDER TRAVERSAL
4. POSTORDER TRAVERSAL
5. DELETE
6. COUNT LEAF NODES
7. COUNT INTERNAL NODES
8. HEIGHT OF THE TREE
9. EXIT
Enter your choice: 9
Program terminated.
PS C:\Users\akhil\Desktop\vscode\python lab programs (dsa)>

Ln 153, Col 33  Spaces: 4  UTF-8  CRLF  { } Python  Python 3.13 (64-bit)
```

WEEK – 11

PROGRAM-11:

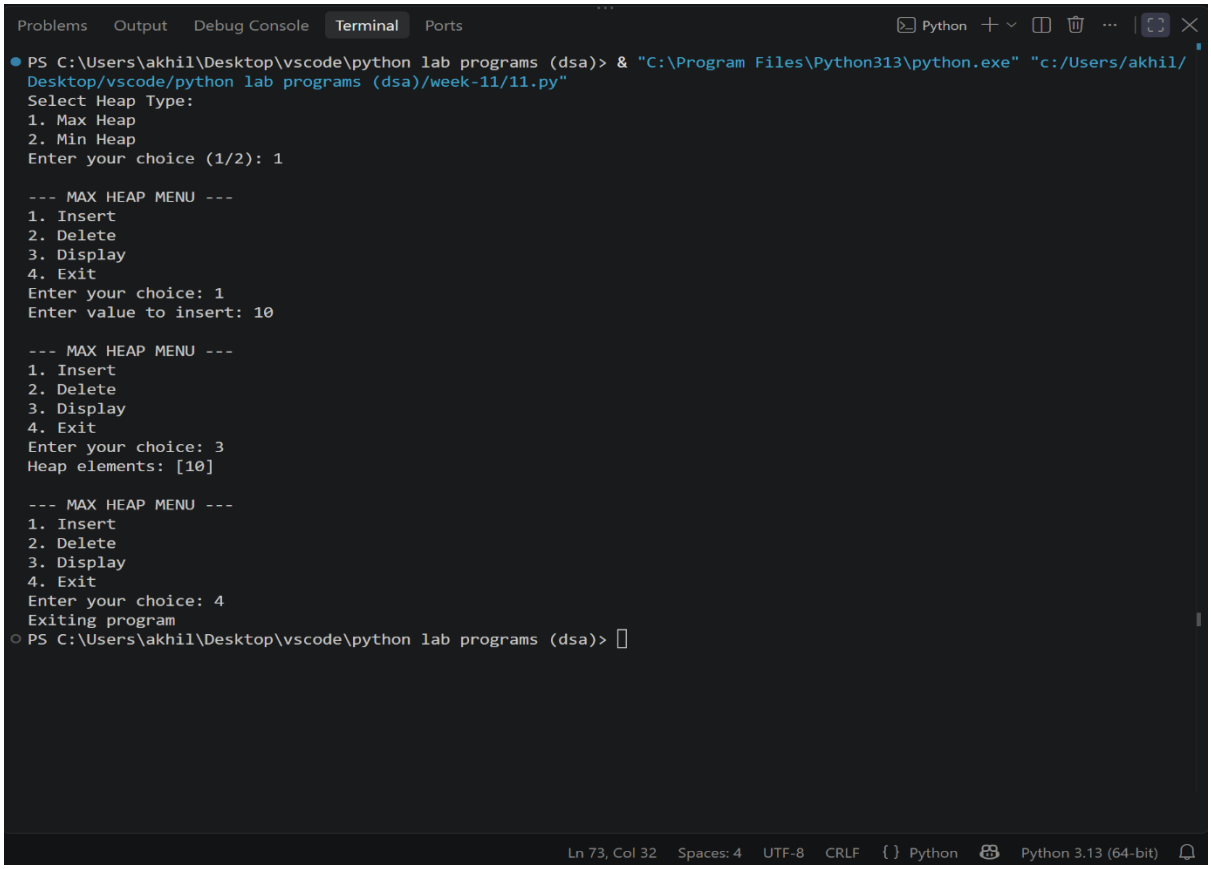
AIM: To write a menu-driven Python program to implement a Binary Heap (Max Heap and Min Heap) and perform Insertion, Deletion of the root element and Display of the array representation of the heap.

CODE:

```
def insert(heap, value, is_max):
    heap.append(value)
    i = len(heap) - 1
    while i > 0:
        parent = (i - 1) // 2
        if (is_max and heap[i] > heap[parent]) or (not is_max and heap[i] <
heap[parent]):
            heap[i], heap[parent] = heap[parent], heap[i]
            i = parent
        else:
            break
def delete(heap, is_max):
    if len(heap) == 0:
        print("Heap is empty")
        return
    print("Deleted element:", heap[0])
    heap[0] = heap[-1]
    heap.pop()
    n = len(heap)
    i = 0
    while True:
        target = i
        left = 2 * i + 1
        right = 2 * i + 2
        if is_max:
            if left < n and heap[left] > heap[target]:
                target = left
            if right < n and heap[right] > heap[target]:
                target = right
        else:
            if left < n and heap[left] < heap[target]:
                target = left
            if right < n and heap[right] < heap[target]:
                target = right
        if target != i:
            heap[i], heap[target] = heap[target], heap[i]
            i = target
        else:
            break
```

```
def display(heap):
    if len(heap) == 0:
        print("Heap is empty")
    else:
        print("Heap elements:", heap)
print("Select Heap Type:")
print("1. Max Heap")
print("2. Min Heap")
while True:
    t = input("Enter your choice (1/2): ")
    if t in ("1", "2"):
        break
    print("Invalid choice")
is_max = (t == "1")
name = "MAX HEAP" if is_max else "MIN HEAP"
heap = []
while True:
    print("\n--- " + name + " MENU ---")
    print("1. Insert")
    print("2. Delete")
    print("3. Display")
    print("4. Exit")
    choice = int(input("Enter your choice: "))
    if choice == 1:
        value = int(input("Enter value to insert: "))
        insert(heap, value, is_max)
    elif choice == 2:
        delete(heap, is_max)
    elif choice == 3:
        display(heap)
    elif choice == 4:
        print("Exiting program")
        break
    else:
        print("Invalid choice")
```

OUTPUT:


Problems
Output
Debug Console
Terminal
Ports
Python
PS C:\Users\akhil\Desktop\vscode\python lab programs (dsa)> & "C:\Program Files\Python313\python.exe" "c:/Users/akhil/Desktop/vscode/python lab programs (dsa)/week-11/11.py"
Select Heap Type:
1. Max Heap
2. Min Heap
Enter your choice (1/2): 1
--- MAX HEAP MENU ---
1. Insert
2. Delete
3. Display
4. Exit
Enter your choice: 1
Enter value to insert: 10
--- MAX HEAP MENU ---
1. Insert
2. Delete
3. Display
4. Exit
Enter your choice: 3
Heap elements: [10]
--- MAX HEAP MENU ---
1. Insert
2. Delete
3. Display
4. Exit
Enter your choice: 4
Exiting program
PS C:\Users\akhil\Desktop\vscode\python lab programs (dsa)>
Ln 73, Col 32
Spaces: 4
UTF-8
CRLF
{ } Python
Python 3.13 (64-bit)

WEEK – 12

PROGRAM-12(A):

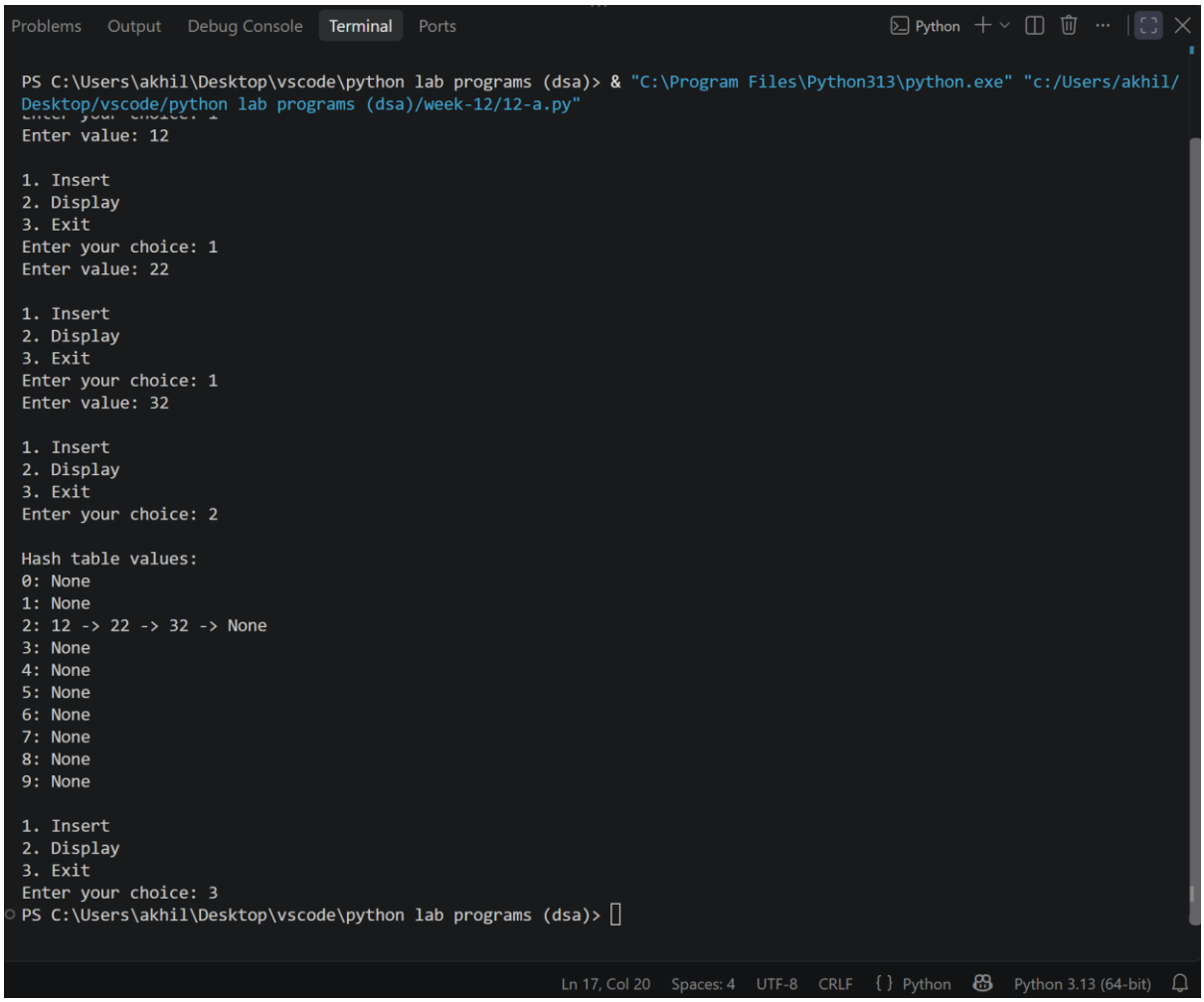
AIM: To write a Python program to implement a Hash Table using Separate Chaining for collision resolution.

CODE:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
SIZE = 10
ht = [None] * SIZE
def insertValue(value):
    index = value % SIZE
    newnode = Node(value)
    if ht[index] is None:
        ht[index] = newnode
    else:
        temp = ht[index]
        while temp.next is not None:
            temp = temp.next
        temp.next = newnode
def displayValue():
    for i in range(SIZE):
        print(i, end=": ")
        temp = ht[i]
        if temp is None:
            print("None")
        else:
            while temp is not None:
                print(temp.data, end=" -> ")
                temp = temp.next
            print("None")
repeat = True
while repeat:
    print("\n1. Insert")
    print("2. Display")
    print("3. Exit")
    ch = int(input("Enter your choice: "))
    if ch == 1:
        value = int(input("Enter value: "))
        insertValue(value)
    elif ch == 2:
        print("\nHash table values:")
        displayValue()
    elif ch == 3:
```

```
        repeat = False
    else:
        print("Invalid choice")
```

OUTPUT:



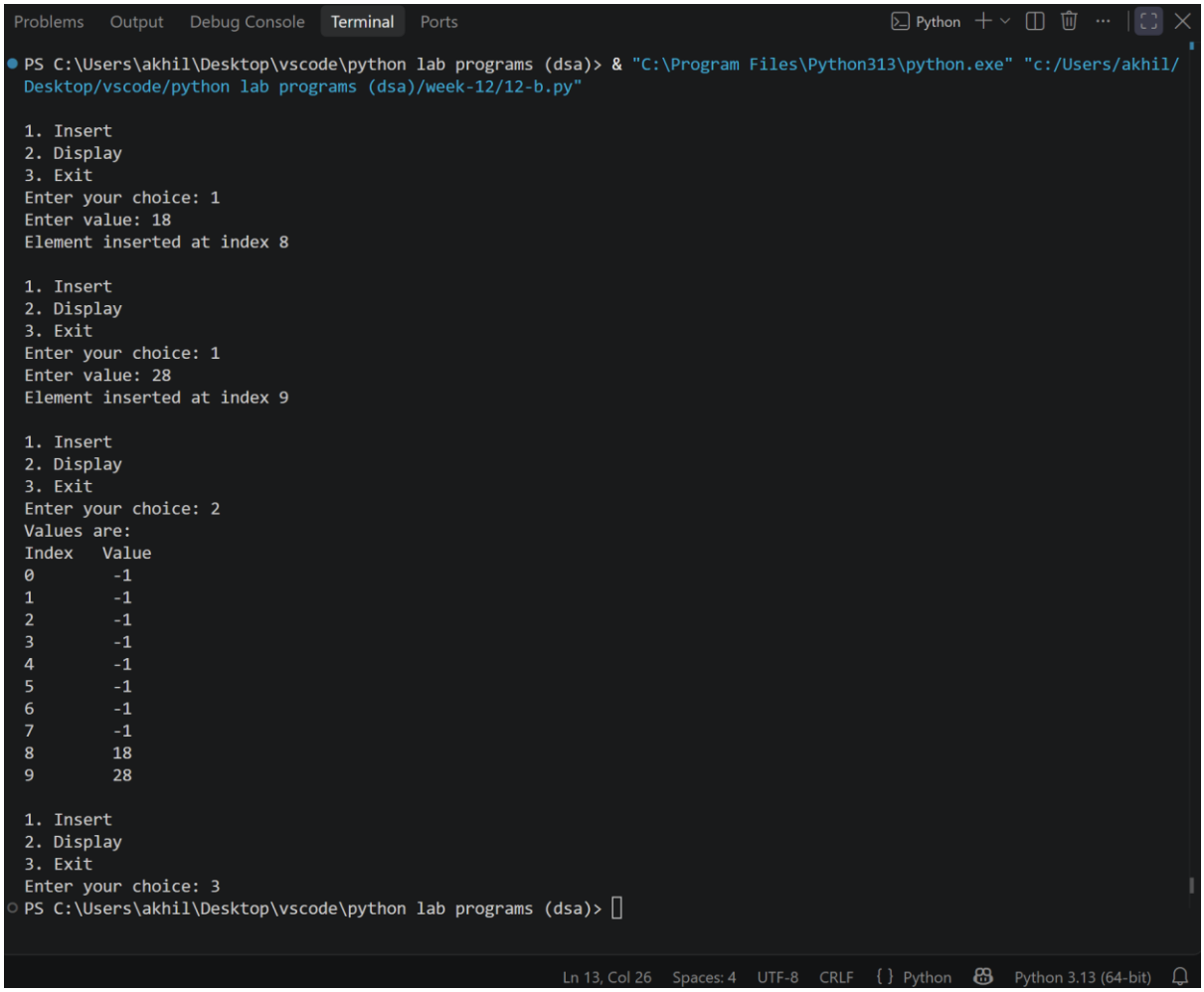
PROGRAM-12(B):

AIM: To write a Python program to implement a Hash Table using Linear Probing for collision resolution.

CODE:

```
SIZE = 10
a = [-1] * SIZE
def insertValue(x):
    for i in range(SIZE):
        index = ((x % SIZE) + i) % SIZE
        if a[index] == -1:
            a[index] = x
            print("Element inserted at index", index)
            return
    print("Hash table is full")
def displayValue():
    print("Index\tValue")
    for i in range(SIZE):
        print(i, "\t", a[i])
repeat = True
while repeat:
    print("\n1. Insert")
    print("2. Display")
    print("3. Exit")
    ch = int(input("Enter your choice: "))
    if ch == 1:
        value = int(input("Enter value: "))
        insertValue(value)
    elif ch == 2:
        print("Values are:")
        displayValue()
    elif ch == 3:
        repeat = False
    else:
        print("Invalid choice")
```

OUTPUT:


Problems
Output
Debug Console
Terminal
Ports
Python
PS C:\Users\akhil\Desktop\vscode\python lab programs (dsa)> & "C:\Program Files\Python313\python.exe" "c:/Users/akhil/Desktop/vscode/python lab programs (dsa)/week-12/12-b.py"
1. Insert
2. Display
3. Exit
Enter your choice: 1
Enter value: 18
Element inserted at index 8
1. Insert
2. Display
3. Exit
Enter your choice: 1
Enter value: 28
Element inserted at index 9
1. Insert
2. Display
3. Exit
Enter your choice: 2
Values are:
Index Value
0 -1
1 -1
2 -1
3 -1
4 -1
5 -1
6 -1
7 -1
8 18
9 28
1. Insert
2. Display
3. Exit
Enter your choice: 3
PS C:\Users\akhil\Desktop\vscode\python lab programs (dsa)>
Ln 13, Col 26
Spaces: 4
UTF-8
CRLF
{ } Python
Python 3.13 (64-bit)

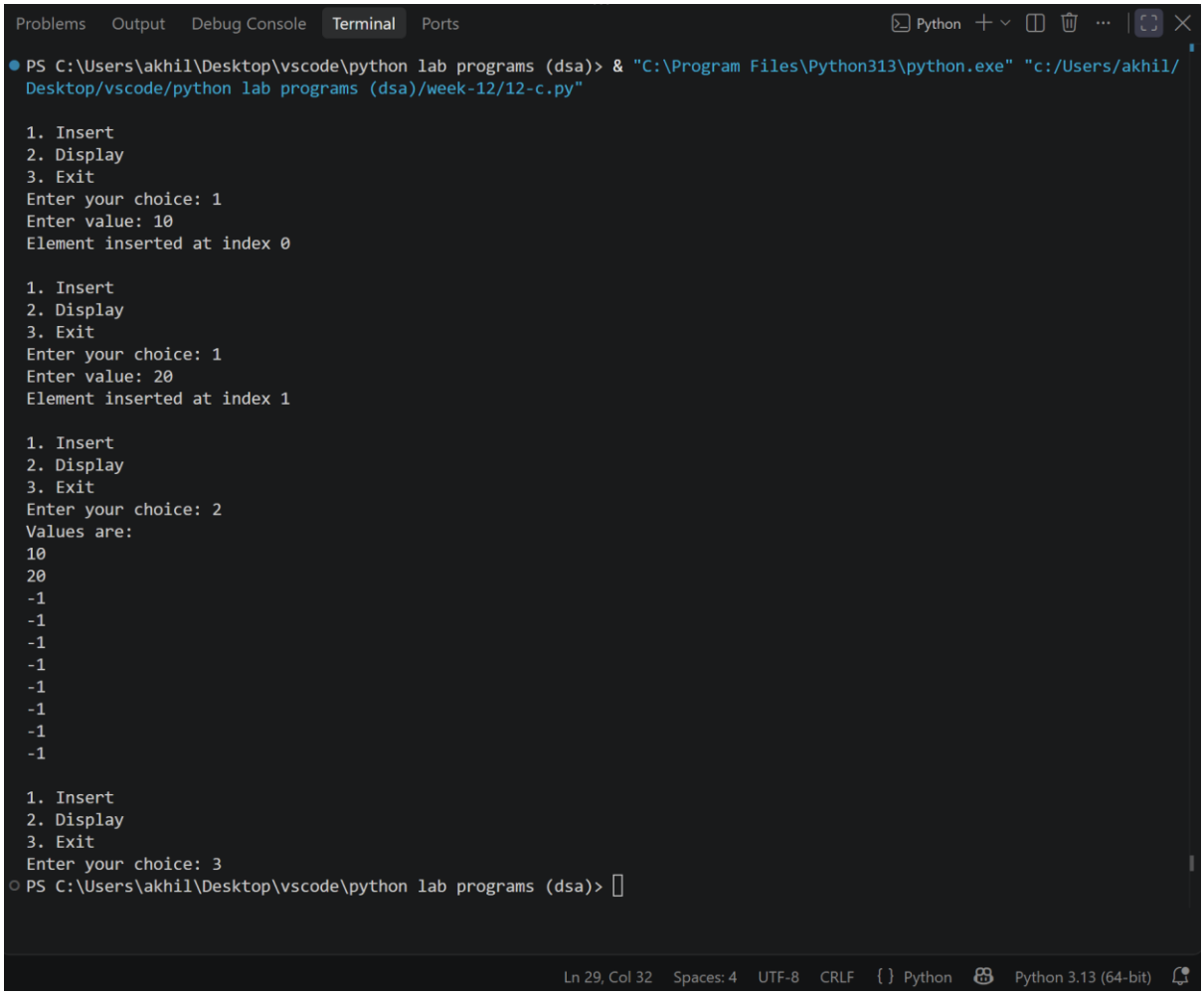
PROGRAM-12(C):

AIM: To write a Python program to implement a Hash Table using Quadratic Probing for collision resolution.

CODE:

```
SIZE = 10
a = [-1] * SIZE
def insertValue(value):
    for i in range(SIZE):
        index = ((value % SIZE) + i * i) % SIZE
        if a[index] == -1:
            a[index] = value
            print("Element inserted at index", index)
            return
    print("No empty slot found in the probe sequence")
def displayValue():
    print("Values are:")
    for i in range(SIZE):
        print(a[i])
repeat = True
while repeat:
    print("\n1. Insert")
    print("2. Display")
    print("3. Exit")
    ch = int(input("Enter your choice: "))
    if ch == 1:
        value = int(input("Enter value: "))
        insertValue(value)
    elif ch == 2:
        displayValue()
    elif ch == 3:
        repeat = False
    else:
        print("Invalid choice")
```

OUTPUT:


Problems
Output
Debug Console
Terminal
Ports
Python
PS C:\Users\akhil\Desktop\vscode\python lab programs (dsa)> & "C:\Program Files\Python313\python.exe" "c:/Users/akhil/Desktop/vscode/python lab programs (dsa)/week-12/12-c.py"
1. Insert
2. Display
3. Exit
Enter your choice: 1
Enter value: 10
Element inserted at index 0
1. Insert
2. Display
3. Exit
Enter your choice: 1
Enter value: 20
Element inserted at index 1
1. Insert
2. Display
3. Exit
Enter your choice: 2
Values are:
10
20
-1
-1
-1
-1
-1
-1
-1
-1
1. Insert
2. Display
3. Exit
Enter your choice: 3
PS C:\Users\akhil\Desktop\vscode\python lab programs (dsa)>
Ln 29, Col 32
Spaces: 4
UTF-8
CRLF
{ } Python
Python 3.13 (64-bit)